Seminário de Estrutura de dados:

Tabela *Hash*

12 de maio de 2015

Seminário sobre as Tabelas *Hash*

apresentado pelas alunas Agnes Lopes Caon,

Bárbara C. Carvalho da Silva

e Crislainde Rocha Machado

ao professor Samuel Basílio

na disciplina de Estrutura de dados.

7 de maio de 2015

Sumário

1 Introdução 4
2. A tabela Hash 4
2.1 Função de dispersão 5
2.2 Formas de tratar as colisões 5
2.2.1 Uso da posição consecutiva livre 5
2.2.2 Uso de uma segunda função hash 6
2.2.3 Uso de lista encadeada 7
3. Tipos de operações na tabela Hash 8
4. Vantagens e desvantagens encontradas na utilização de tabelas Hash 8
5. Aplicação das tabelas Hash 9
6. Implementação do código de uma tabela hash 9
7. Conclusão 16
8. Bibliografia 17

1 Introdução

Com o intuito de armazenar elementos organizadamente, vetores podem ser criados. Porém, dependendo da quantidade de elementos e da flexibilidade que eles necessitem ter para ser inseridos ou retirados, esses vetores podem depender de muita memória, tornando essa estrutura inadequada.

Uma forma de melhorar essa estrutura seria criar um vetor de ponteiros que então só gastaria memória com as células ocupadas, deixando as desocupadas receberem NULL.

Mas ainda assim, existe uma outra estrutura que é capaz de organizar esses elementos obedecendo uma função lógica. Assim é possível busca rápida e fácil gastando o mínimo de memória. Essa estrutura pode ser chamada de *tabela hash, "hash table" ou tabela de dispersão*, que será explorada nesse seminário.

2. A tabela Hash

Hash é uma estrutura de dados do tipo dicionário, ou seja, são estruturas que possibilitam as operações de inserir, pesquisar e remover, utilizando uma função sobre parte da informação (chave) para retornar o índice onde a informação deve ou deveria estar armazenada. Essa função de mapeamento a chave para um índice de um arranjo é chamada de Função Hashing.

Nas tabelas Hash, cada elemento é identificado por um valor (chave de pesquisa) que o identifique de forma eficiente, isso é, evitando gastar memória com valores muito grandes quando o que realmente importa ou faz sentido a uma associação efetiva é um valor muito menor.

Através de uma função de dispersão ou função de Hashing é gerado um índice para determinada chave e assim as informações são distribuídas pela tabela Hash. O ideal então seria que cada índice retornasse a uma única informação, então ela é de fundamental importância para a eficiência da tabela e de suas operações.

Para evitar desperdício de memória com posições vazias em um vetor de estruturas, esse vetor deve ser de ponteiros. Cada elemento pode ser alocado dinamicamente à estrutura dele e armazenar um ponteiro para ela no vetor.

Porém, dependendo dos valores podem haver dois ou mais elementos que através da função de dispersão se encaixem na mesma posição. Quando dois elementos são mapeados para o mesmo índice na tabela é dito que ocorreu uma *colisão* que pode ser tratada para o funcionamento ideal da estrutura.

2.1 Função de dispersão

Como anteriormente já dito, é a função de dispersão que faz referência a um índice da tabela de acordo com a chave inserida para busca, ou seja, passa-se o valor do elemento como parâmetro dessa função/chave de busca e ela retorna a posição/índice da tabela que o ponteiro do elemento se referenciará (no caso de inserção de valores) ou retornará o conteúdo daquela posição (no caso de busca).

Então, para que a função Hash seja o mais eficiente possível ela deve ter as seguintes propriedades:

- Ser eficientemente avaliada de modo que a avaliando já se pode determinar onde o elemento se encontra, ou seja, ser de acesso rápido.
- Espalhar bem a chave de busca, ou seja, utilizar a função de hash que resulte no mínimo de colisões possível para continuar garantindo o acesso rápido, evitando assim os tratamentos adicionais de colisão.

Mesmo que a função garanta o mínimo de colisões, o ideal é que a tabela não tenha uma ocupação maior do que 75% da sua dimensão para garantir uma folga aos elementos armazenados nem menor do que 25% para não desperdiçar memória.

2.2 Formas de tratar as colisões

É comum ocorrerem colisões (a menos que se tratem de funções de Hashing na criptografia, que são as que exigem ser perfeitas), isso acontece geralmente porque o número de chaves possíveis é muito maior do que o número de entradas possíveis da tabela e porque não é possível garantir que as funções de dispersão são eficientes o suficiente para o espalhamento das informações. Sem contar que o paradoxo do aniversário de Fellen já previa que num grupo de 23 pessoas juntas ao acaso já existe mais de 50% de chance de 2 dessas pessoas façam aniversário no mesmo dia, quem dirá em arrays volumosas como as que são frequentemente aplicadas na tabela Hash.

Através do cálculo do fator de carga α (razão entre o número de elementos e o número de índices), as principais medidas para resolver o caso de colisão são pelo endereçamento aberto (como, por exemplo, o uso da posição consecutiva livre, uso de uma segunda função de dispersão, que são métodos que os elementos ficarão referenciados em posições desocupadas, ou seja, não precisam de ponteiros) quando o resultado do fator de carga é $0<\alpha<1$ ou pelo endereçamento encadeado (uso de listas encadeadas, método de criação de lista em uma posição) frequentemente usado quando $\alpha>1$.

Mostrar-se-ão tais métodos de tratamento de colisões.

2.2.1 Uso da posição consecutiva livre

Nessa estratégia, através de um incremento circular, procurar-se-á pela próxima posição livre para armazenar o ponteiro que referencia elemento que estava colidindo. Graças à política de não permitir que uma tabela tenha mais do que 75% dos índices

ocupados justamente para não aumentar a chance de colisão na tabela, sempre ter-se-á alguma posição livre.

Quando o elemento que estava colidindo for procurado através da função hash, ele será procurado a partir da ocorrência da colisão até que seja encontrado ou que uma posição vazia apareça, como mostra a figura 1.

chave	K = ord(chave)	$i_1 = h(K)$	$i_2 = rh(i_1)$	$i_3 = rh(i_2)$	$i_4 = rh(i_3)$
C	67	4	-	-	-
H	72	2	-	-	-
A	65	2	3	-	-
V	86	2	3	4	5
E	69	6	-	-	-
S	83	6	0	-	-

Tabela

0	S
1	
2	H
3	A
4	C
5	V
6	E

Figura 1: esquematização do funcionamento do endereçamento aberto pelo uso da posição consecutiva livre.

É preciso se atentar que a existência de algum elemento já ocupando a posição que o procurado não garante que ele tenha sido alocado e que a partir desse índice mapeado é procurar o valor real do elemento, não mais o valor que foi associado a ele de forma representativa.

Se verificado que o elemento requerido não esteja alocado, deve-se ter uma função que possibilite a inserção dele. Essa função tem o poder de modificar um elemento se ele já existir ou adicioná-lo na ausência do mesmo na posição vazia mais próxima encontrada a partir do índice mapeado.

Embora esse seja um método simples de tratar as colisões, desse modo ocorrerá a concentração de lugares ocupados na tabela enquanto o ideal seria dispersar.

2.2.2 Uso de uma segunda função hash

Método muito parecido com o anterior, porém possui uma segunda função de dispersão para evitar a concentração de lugares ocupados na tabela. Através dessa segunda função para tratar a colisão, procura-se uma posição livre com incrementos circulares dados por essa nova função. Um exemplo do funcionamento desse método é mostrado na figura 2.

Chave (k)	Adams	Jones	Morris	Smith
$h_1(k)$	5	6	6	5
$h_2(k)$	2	3	4	3

0	
1	
2	
3	
4	
5	Adams
6	Jones
7	
8	Smith
9	
10	Morris

Figura 2: esquematização do funcionamento do endereçamento aberto pelo uso de uma segunda função hash.

Cuidados: a segunda função deve ser limitada a não retornar zero (incremento zero não resolveria nada) e de preferência não deve retorna um número divisor da dimensão da tabela também, porque isso limitaria a procura pela posição livre num conjunto restrito dos índices.

2.2.3 Uso de lista encadeada

Diferente das estratégias anteriores em que o elemento que "chegasse primeiro" no índice correspondente ficava com o lugar e o que causasse a colisão teria que encontrar outra posição vazia, nessa estratégia cada elemento da tabela Hash representa um ponteiro para uma lista encadeada, ou seja, todos elementos da tabela formam uma lista adjunta aos respectivos índices e havendo mais de um elemento que corresponda a um índice, eles se juntarão à lista respectiva.

Para a estrutura da informação deve-se colocar um ponteiro adicional para o próximo item da lista, como mostra a figura 3 abaixo.

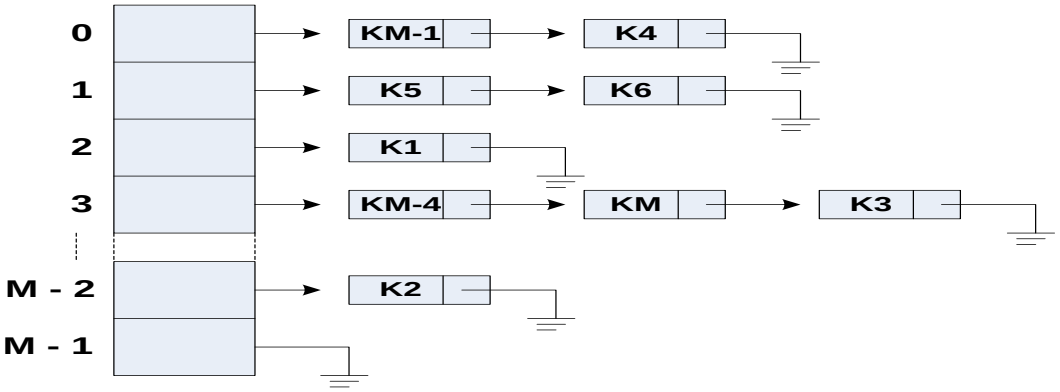

Figura 3: esquematização do funcionamento do endereçamento fechado pelo uso de listas encadeadas.

3. Tipos de operações na tabela Hash

Como especificação da tabela Hash, pode-se fazer nela operações de criação, destruição, status, operações básicas e outras operações.

Operações de criação e destruição são as funções de criar uma tabela inicialmente vazia e a de destruir uma tabela.

Operações de status são funções de verificar se a tabela está vazia, se está cheia e o fator de carga da tabela.

Operações básicas são as operações que já entraram em exemplos já visto, como as funções de inserção de itens, busca e remoção de itens.

Outras operações são as funções de limpeza (todos os itens são descartados) e do tamanho (para saber o número de elementos em uso).

4. Vantagens e desvantagens encontradas na utilização de tabelas Hash

- **Vantagens:**
  - ✓ Podemos adequar o tamanho da tabela hash ao *n* esperado em nossa aplicação;
  - ✓ Eficiência para *n* grandes;
  - ✓ Para trabalharmos com problemas envolvendo *n* = 1.000.000 de dados, podemos imaginar uma tabela de hashing com 2.000 entradas, onde temos uma divisão do espaço de busca da ordem de *n*/2.000 de imediato;
  - ✓ Aplicação imediata a arquivos;
  - ✓ Os métodos de hashing, tanto de endereçamento aberto como fechado, podem ser utilizados praticamente sem nenhuma alteração em um ambiente de dados persistentes utilizando arquivos em disco;
  - ✓ São melhores do que listas quando a quantidade de dados é maior e são estruturas mais simples do que árvores;

- **Desvantagens**
  - ➢ Dependência da escolha de função de hashing;
  - ➢ Para que um tempo de acesso médio ideal seja mantido, é necessário que a função de hashing divida o universo dos dados de entrada em b conjuntos de tamanho aproximadamente igual.
  - ➢ Tempo médio de acesso é ótimo somente em uma faixa;
  - ➢ A complexidade linear implica em um crescimento mais rápido em relação a n do que as árvores, por exemplo;
  - ➢ Existe uma faixa de valores de n, determinada por b, onde o hashing será muito melhor do que uma árvore. Fora dessa faixa é pior;

- Não permite recuperar/imprimir todos os elementos em ordem de chave (como a árvore permite) nem tampouco outras operações que exijam sequência dos dados;
- Não permite operações do tipo recuperar o elemento com a maior ou a menor chave (como é possível nas árvores).

5. Aplicações das tabelas Hash

- Busca de elementos em base de dados como estruturas de dados em memória, bancos de dados, mecanismos de busca na Internet;
- Na verificação de integridade de dados grandes como no envio de dados: com resultado da função *hash* o receptor calcula função *hash* sobre dados recebidos e obtém novo resultado. Se resultados iguais, dados são iguais, se diferentes, novo *download* no Linux, por exemplo, é feito;
- No armazenamento de senhas com segurança: somente resultado função *hash* é armazenado no servidor;
- Hashes criptográficos.

6. Implementação do código de uma tabela hash

```
// Exemplo solucionando possíveis colisões com uso de uma lista
encadeada, compondo a tabela que tem vetores de ponteiros

#include<conio.h>// Inclui as bibliotecas

#include<stdio.h>

#include<stdlib.h>

#include<iostream>

#define tam 4 // Define o tamanho

// Vai existir uma celula e dentro dessa celula vai criar uma
variavel do tipo celula que é o apontador.

//Ponteiro do tipo celula, faz o prox apontar para outros
elementos

 typedef struct celula *apontador; // Cria um apontador


 typedef struct celula

 {

     int chave; // Chave é o valor criado pela função hash para
ir direto no código
```

```
     apontador prox;// irá apontar para os objetos para que se
possa percorrer a lista

 }celula;


 typedef struct tab

 {

     apontador primeiro, ultimo;// Cria o primeiro e o ultimo
para definir os locais

 }tab;


 int Funcao(int num) // Criação da chave para armazenamento dos
valores de acordo com a Função Hash

 {

     return num % tam ;// Função hash

 }


 void criarTabela(tab Tabela[])// Tem que criar a tabela para
armazenar os valores, e vai inserir [] pois é um vetor de
vetores

 {

     int i;// Criação do for para que seja criado espaços de
memoria de acordo com o tamanho definido onde será inserido os
valores, criando então a Tabela

     for(i=0;i<tam;i++)

     {

          Tabela[i].primeiro=(apontador)
malloc(sizeof(celula));// cria um espaço de memoria, Tabela
aponta para o espaço criado

          Tabela[i].ultimo=Tabela[i].primeiro;// Faz o ultimo
apontar para o primeiro, para iniciar com a tabela vazia sem
lixo

          Tabela[i].ultimo->prox=NULL; // O último aponta para
NULL, para demonstrar q a tabela para nesse instante e receba um
NULL e não lixo de memória

     }

 }

 void Insere(tab Tabela[])
```

```
 {
     int i,num,cod;
     for(i=0;i<tam;i++) // o usuário digitara a quantidade de
espaços de memoria criado, obriga o usuário a digitar o tamanho
definido, porém
     {      //evita dele pedir a opção inserir toda vez que
quiser digitar a quantidade de numeros que a tabela permite
inserir . Então opcional
          printf("\n Digite um numero a ser armazenado :\n");
          scanf("%d",&num);
          cod=Funcao(num);// o cod vai receber a chave definida
pela função hash
          Tabela[cod].ultimo->prox = (apontador) malloc (sizeof
(celula));// O ultimo vai apontar para o proximo onde vai ser
criada um espaço de memoria
          Tabela[cod].ultimo=Tabela[cod].ultimo->prox;//o
ultimo recebe o ultimo que aponta para o proximo que é a celula
criada,então o ultimo vai ter uma celula vazia
          Tabela[cod].ultimo->chave=num;//O ultimo portanto
aponta para a chave e recebe o numero digitado
          Tabela[cod].ultimo->prox=NULL; // O último aponta
para o proximo,para demonstrar q a tabela para nesse instante e
aponte para um NULL e não lixo de memória
     }
     system("cls");//Limpa a Tela
}


 void Mostrar(tab Tabela[])
 {
     int z, aux;
     printf("\n--------- TABELA ----------\n");
     for(z=0;z<tam;z++){ apontador aux=Tabela[z].primeiro->prox;
// cria um aux pelo apontador, q recebe em ordem feita pelo for
através da variavél z começando pelo 0
        // e começa no primeiro da tabela e vai apontando para o
próximo
        while(aux!=NULL){ // Enquanto o aux for diferente de
NULL o laço continua, quando for igual o lanço termina pois
chegou o fim da lista
```

```
                printf(" %d ",aux->chave); // aux aponta para a
chave onde está o número armazenado

                aux=aux->prox; // aux recebe o próximo do aux

            }

        printf("\n");

      }

    printf("\n");

 }


 void Pesquisar(tab Tabela[])

 {

      int x, y , z=0 , aux;

      printf("\n Digite a cordenada X (0 < %d)\n",tam);

      scanf("%d",&x);

      printf("\n Digite a cordenada Y (0 < %d)\n",tam);

      scanf("%d",&y);

      system("cls"); //Limpa a Tela

      if(x>=0 && x<tam) // Busca o valor se estiver certo a
coordenada

      {

            apontador aux=Tabela[x].primeiro->prox; // cria um
aux pelo apontador, q recebe em ordem feita pelo for através da
variavél z começando pelo 0

              // e começa no primeiro da tabela e vai apontando
para o próximo

              while(aux!=NULL){ // Enquanto o aux for diferente
de NULL o laço continua, quando for igual o lanço termina pois
chegou o fim da lista


                        if(z==y)// se z for igual a coordenada y
então imprimi o valor que está na chave

                          printf(" %d ",aux->chave); // aux
aponta para a chave onde está o número armazenado

                         z++; // caso z não for igual a y então
recebe +1 até o valor ser igual e imprimir o valor da chave
correspondente
```

```
                    aux=aux->prox; // aux recebe o próximo do
aux
           }
      }
    else
      {
           printf("\n Coordenadas inválidas. \n Tente novamente.
\n");
      }
 }


void Remover(tab Tabela[])
 {
      int x, y ,z=0 ,i=-1;


      printf("\n Digite a cordenada X (0 < %d)\n",tam);
      scanf("%d",&x);


      printf("\n Digite a coordenada Y (0 < %d)\n",tam);
      scanf("%d",&y);
      system("cls"); //Limpa a Tela


      apontador aux = Tabela[x].primeiro;// cria uma variavel
auxiliar que comecará apontando para a primeira posição da lista
      if(x>=0 && x<=tam){// Delimitação para que as coordenadas
sejam entre 0 e tam
                     if(Tabela[x].primeiro==Tabela[x].ultimo){
// verificação se a tabela esta vazia
                              printf("\n ****Erro****  \n
Tabela vazia. \n\n");


                    }
                     else{// comandos para remover
                      while(i<(y-1)&&(aux->prox-
>prox!=NULL)){// i<(y-1) faz o aux ficar uma posição anterior ,
aux->prox->prox!=NULL quando o próximo do que pretende
```

```
                        // excluir for NULL então chegou a
posição correta e acaba o laço

                        aux=aux->prox;// percorre o proximo da
lista do codigo até uma posição anterior a que deseja retirar

                        i++; // incrementa o i para mudar a
posição quando for igual ao (y-1)

                      }

                         apontador z = aux->prox;// cria uma
variavel z, que recebe o aux->prox, pois vai esta no elemento
quee quer eliminar

                         aux->prox = z->prox;// auxiliar passa a
apontar para o elemento proximo ao de se deseja exluir, para que
a lista desloque e não fique um espaço

                         // vazio entre duas celulas e ,assim,
não gere o loop

                         free(z);//exclui elemento

                          }

          }

      else

      {

           printf("\n Não existe essa coordenada. \n Tente
novamente \n");

      }

 }


 int main()

 {

      tab Tabela[tam]; //chama a variave tabela onde foi colocada
a lista encadeada

      criarTabela(Tabela);//Chama a tabela

      int num[100],opc;

      printf("\n ---------- TABELA HASH ------------- \n\n")

    while(opc!=5){

        printf("\nDigite um numero correspondente a opcao
desejada\n\n1- Inserir\n\n2- Mostrar\n\n3- Pesquisar\n\n4-
Remover \n\n5- Sair\n\n\n Digite uma opcao: ");

       fflush(stdin);
```

```
        scanf("%d",&opc);
       system("cls");

        switch(opc){
          case 1 :
          Insere(Tabela);
              break;

          case 2 :
          Mostrar(Tabela);
                   break;

         case 3 :
              Pesquisar(Tabela);
              break;

         case 4 :
              Remover(Tabela);
              break;
       }
    }
    printf("\n\n -----------Fim da Tabela Hash------------------
-- \n\n");
     return 0;
 }
```

7. Conclusão

Percebeu-se a importância da tabela Hash quando se objetiva armazenar informações de modo que elas estejam disponíveis assim que precisarmos e de maneira a não desperdiçar memória.

Percebeu-se também que as limitações encontradas na tabela estão na impossibilidade de armazenar informação iguais (a função busca retorna a informação previamente armazenada ao invés de aceitar a inserção dela novamente), impossibilidade de recuperar os elementos em ordem, na necessidade de conhecer bem a chave a ser utilizada na função de dispersão e na possibilidade de necessitar redimensionar a tabela para assegurar o funcionamento ideal no caso de endereçamento aberto.

O ideal e sonhado por todo programador é que sempre se consiga a tabela Hash sem colisões e com o mesmo número de chaves do que seu.

8. Bibliografia

- ZIVIANI, Nivio. **Transformação de Chave (Hashing)**. Projeto de Algoritmos com implementação em PASCAL e C 2ª edição. p. 178. CENGAGE Learning. São Paulo, 2004.
- CELES - RANGEL, W. e JL. **Tabelas de dispersão.** Estrutura de Dados, capitulo 17. – PUC – RJ.
- COUTO e MENOTI, César F. M e David. **Função Hash Mínima e Perfeita**. Departamento de Ciência da Computação. Universidade Federal de Minas Gerais.
- BUENO, Letícia R. **Hashing**. UFABC.
- **Hashing**. Estruturas de Dados – T.332, capítulo 9. UFSC. Disponível em <http://www.inf.ufsc.br/~ine5384hp/Hashing/ >. Acesso em quinta feira, 7 de maio de 2015.
- JAQUES, Patricia. **Estruturas Avançadas de Dados II (Hash**). UNISINOS.
- **Tabelas Hash.** USP. Disponível em: < http://dcm.ffclrp.usp.br/~augusto/teaching/icii/Hash-Tables-Apresentacao.pdf >. Acesso em 29 de abril de 2015.